FOSE SAFAR

Rapport complet du projet

Plateforme web de recherche, réservation et gestion d'appartements à la Résidence Amwaj, Sidi Rahal.

Date de l'analyse : 14 septembre 2026

Version analysée : commit `f2ac723` du 4 septembre 2026

Référentiel : `appartements` — application Django `webbap`

Public visé : direction de projet, équipe fonctionnelle, développeurs, administrateurs et futurs exploitants

Identité et portée du document

Ce rapport décrit l'état réellement observé du projet : objectifs, utilisateurs, fonctions, règles métier, architecture, modèle de données, sécurité, qualité, installation, exploitation, limites et feuille de route.

Méthode. Analyse statique de l'ensemble du dépôt, lecture des modèles, vues, formulaires, services, routes, gabarits, migrations et tests ; exécution de `manage.py check`, `check --deploy`, contrôle des migrations et exécution de la suite de tests sur une base SQLite neuve. Aucun secret du fichier `.env` ni aucune donnée personnelle n'est reproduit.

Sommaire

1. Synthèse exécutive
2. Contexte, objectifs et périmètre
3. Acteurs et droits
4. Périmètre fonctionnel
5. Parcours utilisateur et cycle de réservation
6. Règles métier
7. Architecture technique
8. Modèle de données
9. Interfaces, e-mails et documents
10. Sécurité, confidentialité et conformité
11. Qualité, tests et état vérifié
12. Installation et configuration
13. Exploitation, maintenance et sauvegardes
14. Risques, limites et recommandations
15. Feuille de route proposée
16. Annexes techniques

Important. Le projet est fonctionnel comme démonstrateur avancé, mais il ne doit pas encaisser de vrais paiements en production dans son état actuel : le formulaire bancaire est simulé et aucune passerelle de paiement n'est

connectée.

1. Synthèse exécutive

FOSE Safar est une application web en français permettant de consulter des appartements, rechercher leur disponibilité, réserver un séjour et suivre son traitement. Une tarification et des règles spécifiques sont prévues pour les clients FM6. Un espace d’administration permet de gérer les logements, les réservations, les paiements et des indicateurs d’activité.

7	72	70	31
applications métier	tests détectés	tests réussis	gabarits HTML

Évaluation globale

Axe	État	Appréciation
Couverture fonctionnelle	Avancée	Le parcours principal, les espaces client/admin, les e-mails, les notifications et l’export sont présents.
Règles métier	Solide	Les contraintes centrales sont regroupées dans les modèles/services, protégées par transactions et largement testées.
Qualité automatisée	Bonne	70 tests réussissent sur SQLite ; les 2 tests de concurrence nécessitent PostgreSQL.
Prêt pour production	À préparer	Paiement réel, sécurité HTTPS, validation FM6, base PostgreSQL, conservation des données et exploitation restent à finaliser.
Documentation	Partielle	Un guide de maintenance et un guide OAuth existent ; README, architecture de déploiement et procédures d’incident restent à formaliser.

Points forts

- séparation claire par domaines Django ;
- modèle utilisateur par e-mail et rôles ;
- règles FM6, chevauchements et quotas testés ;
- transactions et verrous prévus pour PostgreSQL ;
- expiration idempotente et notifications ;
- confirmation e-mail avec PDF ;
- interface responsive et cohérente.

Décisions prioritaires

- choisir et intégrer un prestataire de paiement ;
- définir la preuve officielle d’éligibilité FM6 ;
- stabiliser PostgreSQL et appliquer les migrations ;
- relier ou retirer les périodes de vacances ;
- définir l’hébergement, les sauvegardes et la supervision ;
- valider juridiquement les politiques publiées.

2. Contexte, objectifs et périmètre

2.1 Contexte métier observé

La marque présentée dans la majorité de l’application est **FOSE Safar**. L’offre concerne des appartements dans la Résidence Amwaj à Sidi Rahal, au Maroc, avec piscine, jardins, terrain multisports et parking. L’application distingue le public régulier et les bénéficiaires FM6.

2.2 Problème traité

Le projet centralise un processus auparavant susceptible d’être manuel : présentation des logements, contrôle de disponibilité, application des tarifs, respect des quotas, collecte des politiques, paiement, confirmation, notification et pilotage.

2.3 Objectifs fonctionnels

- présenter la résidence et ses appartements ;
- permettre une recherche fiable par dates ;
- appliquer automatiquement les tarifs et règles selon le type de client ;
- empêcher les doubles réservations et conflits d’un même adhérent ;
- faire accepter les conditions avant le paiement ;
- confirmer et documenter le séjour après paiement ;
- donner à l’administrateur une vue opérationnelle et statistique.

2.4 Périmètre inclus

Inclus	Hors périmètre actuel ou non implémenté
Site public, catalogue, photos, recherche, comptes, OAuth Google, réservations, politiques, paiement simulé, reçu, e-mails, PDF, notifications,	Passerelle bancaire réelle, remboursements, facturation fiscale, synchronisation avec plateformes externes, application mobile native, multilingue, channel manager, ménage/check-in

Inclus	Hors périmètre actuel ou non implémenté
administration, tableau de bord, export Excel.	opérationnel, signature électronique.

2.5 Livrables présents dans le dépôt

- code source Django et migrations ;
- 31 gabarits HTML, feuilles de style et JavaScript ;
- 21 images de présentation ou médias ;
- tests automatisés ;
- exemple de configuration `.env.example` ;
- guide OAuth Google ;
- guide de révision et maintenance HTML/PDF.

3. Acteurs et droits

Acteur	Accès	Capacités principales	Restrictions
Visiteur	Public	Accueil, présentation, confidentialité, inscription, connexion, recherche de logements.	Doit être authentifié pour réserver.
Client régulier	Compte e-mail ou Google	Recherche, réservation hors août, profil, politiques, paiement, reçu, historique, notifications.	Tarif régulier ; août bloqué ; aucun chevauchement personnel.
Client FM6	Compte avec rôle FM6 et matricule	Mêmes fonctions client, tarif préférentiel et accès en août.	5 nuits maximum par réservation et 10 nuits par année civile.
Administrateur	Rôle administrateur ou superutilisateur	Logements, photos, périodes, réservations, paiements, tableau de bord, export Excel, administration Django.	Ne peut pas effectuer de réservation client.

Contrôle d’éligibilité FM6 à revoir. À l’inscription, l’utilisateur peut choisir lui-même « Client FM6 » et fournir n’importe quel matricule non vide et unique. Depuis le profil, l’ajout d’un matricule convertit aussi automatiquement un client régulier en client FM6. Aucun référentiel externe ni validation administrative n’est utilisé.

Mécanismes d’autorisation

- `login_required` protège les pages client ;
- `administrateur_required` renvoie une erreur 403 à un utilisateur authentifié non autorisé ;
- les requêtes sur réservations, reçus et notifications sont filtrées par le propriétaire ;
- les formulaires publics d’inscription excluent explicitement le rôle administrateur.

4. Périmètre fonctionnel

Domaine	Fonctions disponibles	Application
Accueil	Présentation de la résidence, services, galerie, localisation, accès au catalogue.	`core`
Comptes	Inscription, connexion/déconnexion, Google OAuth, profil, modification, mot de passe oublié, consentement confidentialité.	`accounts` + `allauth`
Catalogue	Recherche par dates, pagination 12 éléments, tarif personnalisé, fiche logement et galerie.	`apartments`
Réservation	Création, numéro unique, récapitulatif, historique, acceptation des politiques, annulation avant paiement, expiration à 24 h.	`reservations`
Paiement	Saisie factice de carte, validation de format, confirmation atomique, reçu.	`payments`
Communication	E-mail des politiques, confirmation HTML/texte, PDF joint, alertes d’expiration et badge de non-lus.	`notifications` + `services`
Administration	CRUD logements, ajout de photo, périodes de vacances, listes/filtrage, annulation/suppression, back-office Django.	plusieurs applications
Pilotage	Taux d’occupation, revenu, compteurs, série mensuelle, filtre de période et export XLSX.	`dashboard`

Fonctions de confort et d’interface

- design responsive, menu mobile et animations respectant `prefers-reduced-motion` ;
- messages temporaires fermables ;
- dates minimales contrôlées côté navigateur et côté serveur ;
- réinitialisation de mot de passe par e-mail ;
- chargement optimisé des photos avec `prefetch_related` ;
- interface et messages en français, fuseau `Africa/Casablanca`.

5. Parcours utilisateur et cycle de réservation

5.1 Parcours client nominal

Accueil → choix des dates → logements disponibles → fiche logement → connexion → réservation

→ politiques → paiement → reçu + e-mail/PDF

1. Le client saisit une date d'arrivée et une date de départ.
2. Le service exclut les logements indisponibles ou déjà occupés par une réservation confirmée ou une attente de moins de 24 h.
3. Le détail recalcule le tarif correspondant au rôle du client.
4. La création verrouille le client et le logement, expire les anciennes attentes puis valide toutes les règles.
5. Une réservation `EN_ATTENTE` est créée avec un numéro de 9 caractères au format `aaAA00000`.
6. Les politiques sont envoyées par e-mail ; l'utilisateur doit les accepter.
7. Un paiement simulé par carte fait passer le paiement à `PAYE` et la réservation à `CONFIRMEE`.
8. Un e-mail de confirmation est envoyé avec un PDF de deux pages.

5.2 États de réservation

EN_ATTENTE → paiement + politiques acceptées → CONFIRMEE

EN_ATTENTE → 24 h sans paiement → EXPIREE → Notification interne

EN_ATTENTE → annulation client ou admin → ANNULEE

CONFIRMEE → transition prévue mais non automatisée → TERMINEE

REJETEE : état défini sans flux applicatif actif

5.3 Cas d'erreur gérés

- date passée, date de fin antérieure ou égale au début ;

- appartement indisponible ou conflit de période ;
- double réservation du même client, même sur deux logements différents ;
- règles d'août et quotas FM6 ;
- paiement après expiration, montant incohérent ou politiques non acceptées ;
- annulation après paiement refusée ;
- échec d'envoi e-mail signalé sans annuler le paiement déjà confirmé.

6. Règles métier

#	Règle	Valeur / comportement	Implémentation
R1	Tarif régulier	700,00 MAD par nuit par défaut, configurable par appartement.	`Appartement.prix_par_nuit`
R2	Tarif FM6	500,00 MAD par nuit par défaut, configurable par appartement.	`prix_fm6_par_nuit`
R3	Calcul du séjour	Nombre de nuits × tarif ; départ exclu ; durée strictement positive.	`calculer_prix()`
R4	Août	Toute nuit en août est réservée aux clients FM6.	`chevauche_le_mois_aout()`
R5	Durée FM6	5 nuits maximum par réservation.	`MAX_NUITS_FM6_PAR_RESERVATION`
R6	Quota FM6	10 nuits maximum par année civile, ventilées par année si le séjour franchit le 1er janvier.	`_valider_quota_annuel_fm6()`
R7	Conflit client	Un adhérent ne peut avoir deux réservations actives qui se chevauchent, même sur des logements différents.	`reservations_adherent_en_conflit()`
R8	Conflit logement	Les réservations confirmées et attentes récentes bloquent le logement ; bornes semi-ouvertes.	`is_disponible()`
R9	Délai de paiement	24 heures à compter de la création.	`DELAI_PAIEMENT`
R10	Acceptation	Politiques obligatoires avant paiement ; horodatage conservé.	`accepter_politiques()`
R11	Moyen de paiement	Carte bancaire uniquement.	`Paiement.CARTE`
R12	Annulation	Possible avant paiement ; impossible après confirmation/paiement ; tarif non remboursable.	`annuler()`
R13	Garantie	1 000,00 MAD par défaut, remise à l'arrivée ; paramétrable par environnement.	`RAHAL_STAY_GUARANTEE_AMOUNT`
R14	Horaires	Check-in 15:00, check-out 12:00 par défaut.	variables d'environnement

#	Règle	Valeur / comportement	Implémentation

Écart fonctionnel identifié. Le modèle et les écrans permettent à l'administrateur de créer des `PeriodeVacances`, et l'interface affirme qu'elles réservent la disponibilité aux clients FM6. Pourtant, le moteur de disponibilité ne consulte jamais ces enregistrements : seule la règle fixe du mois d'août est appliquée. Un test confirme même qu'un client régulier peut réserver pendant une telle période. Il faut décider si ces périodes doivent devenir actives ou être retirées.

7. Architecture technique

7.1 Vue d'ensemble

Navigateur → routes Django → vues + formulaires → services métier → modèles / ORM →

PostgreSQL ou SQLite

Sorties : templates/CSS/JS · messages · SMTP/e-mails · Pillow/PDF · openpyxl/Excel

Authentification externe : Google OAuth via django-allauth

7.2 Composants applicatifs

Composant	Responsabilité	Fichiers centraux
`webbap`	Configuration, routage racine, WSGI/ASGI.	`settings.py`, `urls.py`
`core`	Accueil public et politique de confidentialité.	`views.py`, templates
`accounts`	Utilisateur personnalisé, rôles, inscription, session, profil, Google.	`models.py`, `forms.py`, `mixins.py`
`apartments`	Logements, photos, périodes, recherche et administration.	`models.py`, `views.py`
`reservations`	Cycle, statuts, quotas, politiques, expiration et	`models.py`, `views.py`, `policies.py`

Composant	Responsabilité	Fichiers centraux
	commandes.	
`payments`	Paiement simulé, validation et reçu.	`models.py`, `forms.py`, `views.py`
`notifications`	Alertes internes, lecture et compteur global.	`models.py`, `context_processors.py`
`dashboard`	Indicateurs administrateur et export XLSX.	`views.py`, `forms.py`
`services`	Transactions de réservation et génération/envoi de confirmation.	`reservation_services.py`, `confirmation_reservation.py`

7.3 Technologies et versions verrouillées

Technologie	Version	Usage
Python	Environnement observé : 3.14.7	Langage serveur
Django	6.0.7	Framework web, ORM, formulaires, sessions, admin
django-allauth	65.18.0	OAuth Google et comptes sociaux
psycopg	3.3.4	Connexion PostgreSQL
Pillow	11.3.0	Images et génération des confirmations PDF
openpyxl	3.1.5	Export Excel des réservations
python-dotenv	1.2.2	Chargement du fichier `.env`
HTML/CSS/JavaScript	natif	Interface sans framework frontal

7.4 Base de données

Si `DB_NAME` existe, l'application utilise PostgreSQL ; sinon elle utilise `db.sqlite3`. PostgreSQL est le choix attendu pour la production et les garanties de concurrence, car SQLite ne prend pas en charge `SELECT FOR UPDATE`.

8. Modèle de données

8.1 Relations principales

Utilisateur 1—N Réservation N—1 Appartement

Réservation 1—1 Paiement · Réservation 1—N Notification

Appartement 1—N Photo · Appartement 1—N PeriodeVacances

Utilisateur et Appartement : suppression protégée si une réservation existe.

Paiement et Notification : suppression en cascade avec la réservation.

8.2 Dictionnaire synthétique

Entité	Champs essentiels	Contraintes / comportement
`Utilisateur`	e-mail, nom, prénom, téléphone, rôle, matricule, consentement, permissions, dates.	E-mail unique ; matricule unique et obligatoire pour FM6 ; mot de passe haché ; identifiant de connexion = e-mail.
`Appartement`	titre, description, tarifs régulier/FM6, capacité, disponibilité.	Capacité ≥ 1 ; ordre alphabétique ; calcul tarifaire centralisé.
`Photo`	fichier image, légende, indicateur principal.	Plusieurs photos par logement ; aucune unicité imposée à la photo principale.
`PeriodeVacances`	logement, début, fin, libellé.	Fin strictement après début ; actuellement non utilisée par la recherche.
`Reservation`	client, logement, dates, création, politiques, numéro, statut, montant.	Numéro unique ; dates valides ; nombreuses validations métier ; suppression protégée côté client/logement.

Entité	Champs essentiels	Contraintes / comportement
`Paiement`	réservation, montant, méthode, statut, date.	Relation un-à-un ; montant vérifié ; paiement atomique ; carte seulement.
`Notification`	client, réservation optionnelle, catégorie, message, envoi, lecture.	Une seule notification d’expiration par réservation ; ordre antéchronologique.

8.3 Intégrité et concurrence

- contraintes SQL sur les dates de réservation et de vacances ;
- unicité SQL des e-mails, matricules et numéros de réservation ;
- transactions atomiques pour créer/modifier une réservation et effectuer un paiement ;
- verrouillage du client puis des appartements dans un ordre déterministe ;
- expiration conditionnelle et notification idempotente ;
- les contrôles de disponibilité complexes restent applicatifs et doivent donc être exécutés par les services prévus.

9. Interfaces, e-mails et documents

9.1 Pages principales

Espace	Pages	Caractéristiques
Public	Accueil, catalogue, détail, confidentialité, inscription, connexion.	Galeries de la résidence, carte/localisation, formulaire de dates, design responsive.
Client	Profil, réservation, politiques, paiement, reçu, mes réservations, notifications.	Récapitulatif, statuts, actions guidées et messages.
Administrateur	Tableau de bord, logements, périodes, réservations, paiements, admin Django.	Filtres, tableaux, création/modification/suppression et export.

9.2 Communication automatique

- **Après création :** e-mail HTML + texte contenant les politiques et un lien d’acceptation.
- **Après paiement :** e-mail HTML + texte avec confirmation et PDF joint.
- **Après expiration :** notification interne unique ; pas d’e-mail d’expiration actuellement.
- **Mot de passe oublié :** e-mail standard Django personnalisé par gabarits.

9.3 Confirmation PDF client

Le service Pillow génère un PDF de deux pages au format image : confirmation, séjour, client et tarif sur la première ; politiques, garantie, horaires et coordonnées sur la seconde. Le document est attaché à l’e-mail, mais aucune route de téléchargement autonome n’est proposée dans l’espace client.

9.4 Tableau de bord

- période par défaut : 30 derniers jours jusqu'au lendemain de la date courante ;
- taux d'occupation calculé sur les logements marqués disponibles ;
- revenu fondé sur la date de paiement, alors que les réservations sont filtrées par chevauchement de séjour ;
- compteurs total, confirmées et annulées ;
- série du nombre de réservations par mois d'arrivée ;
- export Excel des réservations de la période.

10. Sécurité, confidentialité et conformité

10.1 Protections présentes

- hachage des mots de passe via Django et quatre validateurs de robustesse ;
- jetons CSRF sur les formulaires et mutations limitées en POST ;
- sessions et authentification Django ;
- contrôles de rôle et de propriété des objets ;
- protection anti-framing via `XFrameOptionsMiddleware` ;
- clés, identifiants OAuth et SMTP chargés depuis l'environnement ;
- `.env`, base SQLite, médias générés et environnement virtuel ignorés par Git ;
- les numéros de carte saisis ne sont pas stockés dans les modèles.

10.2 Politique de confidentialité

Une page publique décrit les données collectées, une conservation maximale annoncée de cinq ans après la dernière activité et les droits d'accès, rectification, opposition et suppression selon la loi marocaine n° 09-08. L'inscription exige un consentement horodaté.

Éléments juridiques non finalisés. La page utilise à plusieurs endroits la marque « FOSEH Safar » au lieu de « FOSE Safar », mentionne le domaine `www.fosehsafar.com` et laisse le numéro CNDP « à compléter ». La règle de conservation annoncée n'est pas automatisée dans le code. Une validation juridique et métier est nécessaire avant publication officielle.

10.3 Résultat du contrôle de déploiement

`manage.py check --deploy` a signalé six avertissements avec les valeurs de développement : HSTS absent, redirection HTTPS absente, clé secrète de développement, cookies de session non sécurisés, cookie CSRF non sécurisé et `DEBUG=True`.

Réglage recommandé en production	Objectif
`DEBUG=False` et `ALLOWED_HOSTS` explicite	Ne pas exposer les détails internes.
Clé `SECRET_KEY` longue, aléatoire et hors dépôt	Protéger signatures, sessions et jetons.

Réglage recommandé en production	Objectif
`SECURE_SSL_REDIRECT=True`	Forcer HTTPS.
`SESSION_COOKIE_SECURE=True`, `CSRF_COOKIE_SECURE=True`	Empêcher l'envoi des cookies en HTTP.
`SECURE_HSTS_SECONDS` après validation HTTPS	Imposer HTTPS au navigateur.
Proxy de confiance et `SECURE_PROXY_SSL_HEADER` si nécessaire	Reconnaître correctement HTTPS derrière un reverse proxy.

11. Qualité, tests et état vérifié

11.1 Mesures du dépôt

87	4 579	1 311	13
fichiers Python	lignes Python	lignes de tests	migrations métier

Mesures incluant modèles, vues, formulaires, migrations, tests et fichiers d'initialisation ; dépendances du dossier `venv` exclues.

11.2 Résultats exécutés le 14 septembre 2026

Contrôle	Résultat	Interprétation
`manage.py check`	Réussi, 0 problème	Configuration applicative cohérente.
`makemigrations --check --dry-run`	« No changes detected »	Les modèles et fichiers de migration sont synchronisés.
Suite de tests sur SQLite neuve	72 trouvés ; 70 réussis ; 2 ignorés ; 0 échec	Le comportement fonctionnel couvert est stable sur une base fraîche.
Tests ignorés	2 tests de concurrence	SQLite ne prend pas en charge `SELECT FOR UPDATE` ; ils doivent être exécutés sur PostgreSQL.
`check --deploy`	6 avertissements	Réglages de sécurité de production à ajouter.

11.3 Couverture fonctionnelle des tests

Les tests couvrent notamment : inscription/consentement/matricule, OAuth Google, recherche et requêtes optimisées, règles tarifaires, août, quotas FM6, chevauchements, modification via service, politiques, paiement, annulation, expiration, notifications, pages principales et concurrence PostgreSQL.

11.4 État des bases locales

Le fichier `.env` sélectionne PostgreSQL, mais le serveur local sur `localhost:5432` n'était pas joignable pendant l'analyse. La base SQLite de repli est vide et possède huit migrations métier non appliquées. Elle ne doit pas être considérée comme une base opérationnelle à jour. Les tests ont réussi sur une base SQLite temporaire entièrement migrée.

12. Installation et configuration

12.1 Prérequis

- Python compatible avec Django 6.0 ; environnement observé : Python 3.14.7 ;
- pip et environnement virtuel ;
- PostgreSQL recommandé en exploitation ; SQLite possible pour une démonstration locale ;
- compte SMTP pour les vrais e-mails ;
- projet Google Cloud si la connexion Google est activée.

12.2 Installation locale

```
cd /home/iliane/Bureau/appartements

python -m venv venv

source venv/bin/activate

pip install -r requirements.txt

cp .env.example .env

# compléter .env sans publier ses secrets

python manage.py migrate

python manage.py createsuperuser

python manage.py runserver
```

Sans variables `DB_NAME`, l'application utilise automatiquement `db.sqlite3`. Après démarrage, l'accueil se trouve normalement sur `http://127.0.0.1:8000/` et l'administration Django sur `/admin/`.

12.3 Variables d'environnement

Groupe	Variables	Rôle
Django	`SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS`	Sécurité et hôtes.

Groupe	Variables	Rôle
Base	`DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT`	Active PostgreSQL.
Google	`GOOGLE_OAUTH_CLIENT_ID`, `GOOGLE_OAUTH_CLIENT_SECRET`	Connexion sociale ; bouton désactivé si absent.
E-mail	`EMAIL_BACKEND`, `EMAIL_HOST`, `EMAIL_PORT`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`, `EMAIL_USE_TLS`, `DEFAULT_FROM_EMAIL`	SMTP et expéditeur.
Établissement	`RAHAL_STAY_NAME`, `ADDRESS`, `COORDINATES`, `CONTACT_EMAIL`, `CONTACT_PHONE`	Marque et coordonnées des messages/PDF.
Séjour	`RAHAL_STAY_CHECK_IN`, `CHECK_OUT`, `GUARANTEE_AMOUNT`	Politiques et confirmation.

12.4 OAuth Google

Le callback local attendu est `/accounts/google/login/callback/`. Les origines et URI de redirection doivent être créées dans Google Cloud en HTTP local et HTTPS pour le domaine de production.

13. Exploitation, maintenance et sauvegardes

13.1 Tâches récurrentes

Tâche	Fréquence proposée
☐ Exécuter `python manage.py expirer_reservations`.	Toutes les 5 à 15 minutes via cron/planificateur.
☐ Sauvegarder PostgreSQL et vérifier une restauration.	Quotidien ; test mensuel.
☐ Sauvegarder le dossier `media/`.	Quotidien ou stockage objet versionné.
☐ Surveiller erreurs applicatives, SMTP et latence.	Continu.
☐ Contrôler les certificats HTTPS et secrets.	Automatique + revue trimestrielle.
☐ Mettre à jour les dépendances après tests.	Mensuel ou selon alertes sécurité.

13.2 Procédure de livraison recommandée

1. sauvegarder base et médias ;
2. déployer le code dans une version immuable ;
3. installer les dépendances verrouillées ;
4. exécuter les tests sur PostgreSQL ;

5. appliquer `python manage.py migrate` ;
6. exécuter `python manage.py collectstatic --noinput` ;
7. redémarrer les processus WSGI/ASGI ;
8. vérifier accueil, authentification, recherche, réservation de test, e-mail et administration ;
9. surveiller les journaux et indicateurs après livraison.

13.3 Architecture de production minimale

HTTPS / reverse proxy → serveur WSGI/ASGI → Django → PostgreSQL

+ stockage persistant des médias + SMTP + planificateur d'expiration + sauvegardes + supervision

13.4 Observabilité à ajouter

- journalisation structurée avec identifiant de requête et niveau d'erreur ;
- outil de suivi des exceptions ;
- métriques : réservations créées/confirmées/expirées, taux d'échec SMTP et paiement, latence ;
- endpoint de santé vérifiant application et base ;
- alertes sur sauvegarde manquée, base indisponible et accumulation d'attentes expirées.

14. Risques, limites et recommandations

Priorité	Constat	Impact	Recommandation
Critique	Paiement simulé : le format de carte suffit à marquer la réservation payée.	Impossible d'encaisser ou de prouver un paiement réel.	Intégrer une passerelle conforme, utiliser webhooks signés, idempotence, références prestataire et aucun stockage de carte.
Critique	Réglages de production HTTPS et secrets non définis dans le code.	Sessions, CSRF et informations internes pourraient être exposés.	Ajouter une configuration de production séparée et satisfaire `check --deploy`.
Élevée	Éligibilité FM6 auto-déclarée.	Accès indu au tarif 500 MAD et aux dates d'août.	Valider le matricule contre une source autorisée ou introduire un statut « en attente de validation » approuvé par un administrateur.
Élevée	PostgreSQL local indisponible et SQLite de repli non migrée.	Démarrage/tests avec l'environnement courant impossibles ; risque de	Rétablir PostgreSQL, vérifier les sauvegardes, lancer toutes les

Priorité	Constat	Impact	Recommandation
		divergence de schéma.	migrations et la suite complète.
Élevée	`PeriodeVacances` n'influence pas la disponibilité.	Écran administrateur trompeur et règles attendues non appliquées.	Connecter le modèle au moteur de recherche/création ou supprimer la fonctionnalité.
Élevée	Politique juridique incomplète et conservation non automatisée.	Risque de non-conformité et d'information incorrecte.	Faire valider le texte, compléter CNDP/domaine/contact et créer une politique technique de purge/anonymisation.
Élevée	Suppression admin d'une réservation efface en cascade paiement et notifications.	Perte de traçabilité comptable et opérationnelle.	Remplacer par archivage logique, journal d'audit et permissions renforcées.
Moyenne	États `TERMINEE` et `REJETEE` sans transition applicative.	Historique et indicateurs incomplets.	Ajouter un job de clôture et formaliser le cas de rejet, ou retirer les états inutiles.
Moyenne	Plusieurs photos peuvent être marquées principales.	Choix visuel ambigu.	Ajouter une contrainte conditionnelle ou une normalisation lors de l'enregistrement.
Moyenne	Une photo invalide peut être ignorée après sauvegarde du logement.	L'utilisateur croit avoir ajouté une image.	Valider logement et photo avant sauvegarde/redirect et afficher les erreurs.
Moyenne	Pas de pipeline CI, conteneur, manifeste de service ni procédure de retour arrière observés.	Livraisons manuelles et reproductibilité limitée.	Ajouter CI sur PostgreSQL, analyse sécurité, artefact, migration contrôlée et rollback documenté.
Moyenne	Revenu filtré par date de paiement, occupation par dates de séjour.	Une même période du tableau de bord mélange deux conventions.	Afficher explicitement les définitions ou proposer deux filtres distincts.
Moyenne	Marque FOSE/FOSEH incohérente.	Confusion contractuelle et visuelle.	Définir le nom légal et centraliser toutes les occurrences.

15. Feuille de route proposée

Phase 1 — Sécuriser et stabiliser avant production

1. rétablir PostgreSQL, appliquer les migrations et exécuter les 72 tests dont les tests de concurrence ;
2. ajouter la configuration HTTPS/production, secrets robustes et hébergement cible ;
3. remplacer le paiement simulé par une passerelle et tester les webhooks ;

4. mettre en place une validation officielle FM6 ;
5. corriger la politique de confidentialité et la cohérence de marque ;
6. déployer sauvegardes, expiration planifiée et supervision.

Phase 2 — Fiabiliser l’exploitation

1. archivage logique et audit des actions administrateur ;
2. pipeline CI/CD avec PostgreSQL et contrôle de sécurité ;
3. gestion cohérente des périodes de vacances ;
4. automatisation de fin de séjour et de conservation des données ;
5. téléchargement du PDF depuis l’espace client et renvoi d’e-mail contrôlé.

Phase 3 — Enrichir le produit

1. calendrier visuel et recherche par capacité/équipements ;
2. validation FM6 intégrée et espace agent ;
3. factures, avoirs et éventuelle politique de remboursement ;
4. tableau de bord clarifié, export financier et suivi de conversion ;
5. accessibilité auditée, multilingue et performance images.

> **Critère de passage en production proposé.** 72/72 tests réussis sur PostgreSQL, aucun avertissement critique de `check --deploy`, paiement de bout en bout validé en environnement de test, restauration de sauvegarde testée, texte juridique approuvé et procédure d’incident disponible.

16. Annexes techniques

16.1 Arborescence utile

```
appartements/

├── webbap/                # configuration et routes racine

├── core/                  # accueil et confidentialité

├── accounts/              # utilisateurs, rôles, authentification

├── apartments/            # logements, photos, recherche

├── reservations/          # réservation, règles, politiques, expiration

├── payments/              # paiement simulé et reçu

├── notifications/         # alertes internes
```

```
├── dashboard/              # statistiques et export Excel
├── services/               # services transactionnels, e-mail et PDF
├── templates/base.html     # structure commune
├── static/                 # CSS, JavaScript, images publiques
├── media/                  # images téléversées
├── docs/                   # documentation
├── requirements.txt        # dépendances verrouillées
├── .env.example            # variables attendues
└── manage.py
```

16.2 Carte des routes métier

Préfixe	Routes principales	Accès
`/`	accueil, politique de confidentialité	Public
`/accounts/`	inscription, connexion, déconnexion, profil, modification, réinitialisation, routes allauth/Google	Public/client selon page
`/appartements/`	liste, détail, gestion, créer/modifier/supprimer, périodes de vacances	Public/client ; gestion admin
`/reservations/`	réserver, mes réservations, politiques, détail, annuler, gestion	Client ; gestion admin
`/paiements/`	paiement d'une réservation, reçu, gestion	Client propriétaire ; gestion admin
`/notifications/`	liste, marquer lue	Utilisateur authentifié propriétaire
`/tableau-de-bord/`	indicateurs, export XLSX	Administrateur
`/admin/`	administration Django	Staff/superutilisateur

16.3 Commandes de contrôle

```
# Contrôles courants

python manage.py check
```

```
python manage.py check --deploy

python manage.py makemigrations --check --dry-run

python manage.py showmigrations

python manage.py test



# Exploitation

python manage.py migrate

python manage.py collectstatic --noinput

python manage.py expirer_reservations

python manage.py createsuperuser
```

16.4 Checklist de recette manuelle

- ☐ Créer un client régulier puis un client FM6 validé.
- ☐ Vérifier 700/500 MAD et le calcul multi-nuits.
- ☐ Contrôler août, maximum 5 nuits et quota annuel de 10 nuits.
- ☐ Tenter un chevauchement de logement et un chevauchement du même client.
- ☐ Créer une réservation, recevoir les politiques et les accepter.
- ☐ Effectuer un paiement de test et vérifier reçu, statut et PDF.
- ☐ Vérifier qu’une réservation payée ne peut plus être annulée.
- ☐ Faire expirer une attente et contrôler disponibilité + notification unique.
- ☐ Tester la séparation client/admin et l’accès aux objets d’un autre client.
- ☐ Comparer les indicateurs et l’export Excel aux données sources.
- ☐ Tester sur mobile, clavier, lecteur d’écran et mouvement réduit.

16.5 Glossaire

FM6	Catégorie de client bénéficiant de règles et tarifs spécifiques dans l’application.

Bornes semi-ouvertes	L’arrivée est incluse et le départ exclu ; un départ et une nouvelle arrivée le même jour ne se chevauchent pas.
ORM	Couche Django qui manipule la base avec des objets Python.
Migration	Fichier versionné décrivant une évolution du schéma de base.
Idempotent	Une opération répétée ne crée pas d’effet supplémentaire indésirable.
OAuth	Protocole utilisé pour se connecter avec Google sans fournir son mot de passe Google au site.

Conclusion

FOSE Safar dispose d’une base fonctionnelle et technique sérieuse, notamment sur la réservation, les quotas et la cohérence transactionnelle. La priorité n’est plus d’ajouter beaucoup de fonctionnalités, mais de fermer les écarts entre démonstrateur et service exploitable : paiement réel, sécurité de production, preuve FM6, PostgreSQL, conformité et opérations. Une fois ces points traités, le projet pourra évoluer sur une base structurée et bien testée.

Rapport produit automatiquement à partir du dépôt local, sans inclusion de secrets ni de données personnelles. Les constats décrivent la version observée au 14 septembre 2026 et doivent être réévalués après toute modification significative du code ou de l’environnement.